

CYBR 437: Secure Coding

Format String Vulnerabilities

Instructor: Abinash Borah

Outline

- Format String Vulnerabilities
- Another ROP Chain Example
- Discussion (*what is it about?*)

Format String Vulnerabilities

- A format string vulnerability in C occurs when user-controlled input is used directly as the format string in functions like:
 - `printf()`
 - `fprintf()`
 - `sprintf()`
 - `snprintf()`
 - `scanf()` family (less common but related)
- Vulnerability arises when a function expects a format string (containing characters like `%s`, `%d`, `%x`) but receives user input other than regular strings - causing the function to read unintended arguments from the stack

Vulnerable Code

```
char name[10];  
scanf("%9s", name);  
printf(name);           // VULNERABLE  
printf("%s", name);     // SAFER
```

- The user can control the format string
- If the user enters %s%s%s – the vulnerable printf interprets the input string as a reference to string pointers
- Every %s will be interpreted as a pointer to a string – starting from the location of the buffer on the stack
- At some point, it will get to an invalid address and attempting to access it will cause the program to crash

Vulnerable Code (contd.)

- An attacker can also use this to get information from the stack – not just crashing the program

- For example, entering a string

```
Hello %p %p %p %p
```

- Can print something like

```
Hello 000e133e 000e133e 0057f000 cccccccc
```

- The values printed after 'Hello' are values on the stack at that time

Why is it vulnerable?

- The `printf()` reads the format string and for every format specifier it fetches an argument from the stack
- So, if the format string is controlled but no arguments are provided, `printf()` still tries to fetch the arguments from the stack
- And starts reading
 - Saved registers
 - Return addresses
 - Stack canaries
 - Function pointers, etc.

Exploitation Possibilities

- Memory leak
 - If an attacker can see the output of the format function, they can use it to display the stack, since the function can be controlled with the input
- Reading data from arbitrary memory addresses
 - Reading the stack can be interesting for an attacker
 - However, reading arbitrary memory is more valuable for attacks
 - For example, an attacker might reveal the value of a particular secret variable with format string vulnerabilities
- Writing data to arbitrary memory addresses
 - An attacker can use the formatting parameter %n that writes the number of bytes already written by the formatting function to the corresponding pointer
 - If an attacker controls the format string and the target address, they can overwrite return addresses, GOT entries, and function pointers for remote code execution

Differences from Buffer Overflow:

Buffer Overflow	Format String Vulnerabilities
Overwrites memory	Exploits printf()-family parsing
Requires a large input	A small input is sufficient
Corrupts adjacent memory locations	Can read and write arbitrary memory addresses
Rely on unsafe functions	Uses the printf()-family functions